

SCS 1309

Database Management Systems

Lecture 5 – Structured Query Language (SQL)

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

Outline

Database Routines

- Stored Procedures
- Stored Functions

Database Routines

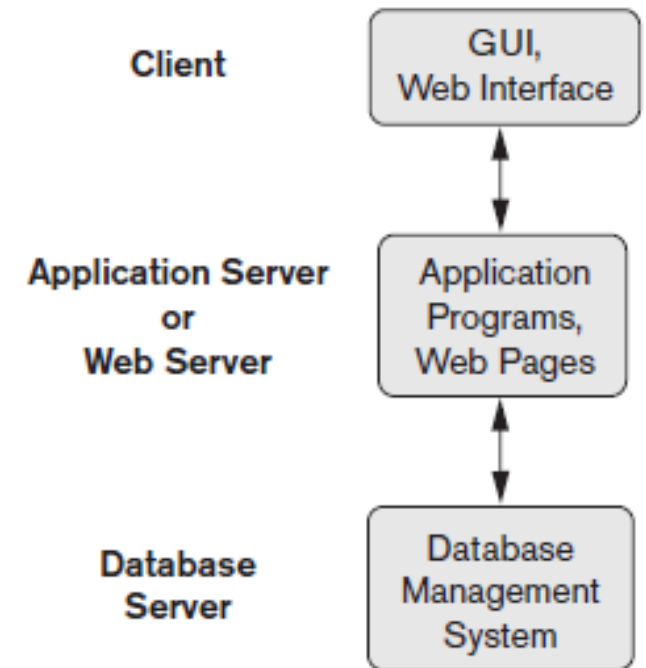
Introduction

- Typically, when working with a relational database, we use individual SQL statements to retrieve or manipulate data, like SELECT, INSERT, UPDATE, or DELETE, directly from within our application code.
- Those statements **work on** and **manipulate** underlying database tables directly.
- If the same SQL statements or a group of SQL statements are used within **multiple applications** accessing the **same database**, they are often duplicated in individual applications.

Example: Consider an enterprise application that always needs to perform specific tasks such as database cleanup, processing payroll, and many more on the database regularly. Such tasks involve multiple SQL statements for executing each task. This process might be easy if we group these tasks into a single task.

Introduction

- **Three-tier client-server architecture:** The database application program is running on a **client machine**, or more likely at the **application server computer** in the middle-tier.
- In either case, the machine where the program is executing is different from the machine on which the database server (the main part of the DBMS software package) is located.
- Although this is suitable for many applications, it is sometimes useful to create **database program modules (procedures or functions)** that are **stored and executed** by the DBMS at the database server.
- These are known as database **routines**, although they can be **functions or stored procedures**.



The **three-tier architecture** adds an intermediate layer between the client and the database server.

Routines

- SQL-invoked **routines** can be either **procedures** or **functions**.

The terms procedure and function are used in the same manner as they are in other programming languages.

- A **function** **returns one value and has only input parameters**.

Example: many built-in functions are included in SQL (SUM(), CONCAT(), UPPER(),CURDATE())

- A **procedure** is a **collection of procedural and SQL statements** that are assigned a **unique name** within the schema and stored in the database.
 - May have input parameters, output parameters, and parameters that are both input and output parameters.
 - A procedure can access database contents and may have local variables.
 - Contain one or more instructions laid out and then executed by the database in a sequence when called.

Routines

- SQL products had developed their own versions of **routines** before the issuance of SQL:1999, so be sure to become familiar with the syntax and capabilities of any product you use.

Database	Stored Procedures	Language Used
Oracle	Yes	PL/SQL
MySQL	Yes	MySQL procedural SQL
SQL Server	Yes	Transact-SQL (T-SQL)
PostgreSQL	Yes	PL/pgSQL
SQLite		Not supported

SQL/PSM (SQL/Persistent Stored Modules)

- The term used in the SQL standard for [routines](#) is **Persistent Stored Modules (PSM)** because these programs are stored persistently by the DBMS, similarly to the persistent data stored by the DBMS.
- Extensions to SQL that are specified in the standard to include [general-purpose programming language constructs](#) in SQL, such as [conditional statements](#) and [loops](#).
- SQL/PSM can be used to write stored procedures and functions.

Why Routines?

The following are some of the advantages of SQL-invoked **routines**:

- **Flexibility:** Unlike constraints or triggers, which are limited to data modification, routines can be invoked in any situation and offer the most robust coding options.
- **Efficiency:** Routines can be crafted and optimized to run more **quickly than generic SQL** statements. Executing a program at the server can **reduce data transfer and communication costs** between the client and server in certain situations.
- **Shareability:** **Centralized storage** at the server allows multiple applications to invoke the same program, improving modularity and reducing duplicate effort.
- **Applicability:** Routines are stored as part of the database and **may apply to the entire database** rather than be limited to one application.
- **Extensibility:** Enhance views with complex types of derived data, and enforce complex constraints that are beyond the specification power of assertions and triggers.

Stored Procedure

Creating a Stored Procedure

- The general form of declaring stored procedures in [SQL/PSM](#) is as follows:

```
CREATE PROCEDURE <procedure_name> (<parameters>)  
BEGIN  
    DECLARE <variable_name> <datatype>; // local declarations  
    <procedure body> ;  
END;
```

- The [<parameters>](#) and [local declarations](#) are optional.
- Declared variables ([local declarations](#)) process retrieved data directly within the procedure.
- [<procedure body>](#) may include,
 - Standard SQL (INSERT, UPDATE) statements
 - PSM control structures like IF-THEN-ELSE or WHILE loops.
 - Error handling instructions (returning error messages to the database users)
 - Calls to other objects in the database

Creating a Stored Procedure

- Since **routines** are stored persistently by the DBMS, it should be possible to call them from the various SQL interfaces and programming techniques.
- The **CALL statement** in the SQL standard can be used to **invoke** a stored procedure (either from an interactive interface or from embedded SQL)

CALL <procedure or function name> (<argument list>) ;

- When it is called, all of the statements in the procedure will be executed.
- This characteristic of procedures helps reduce network traffic because all statements are transmitted at once rather than sent individually.

Show and Drop Procedures

Show Procedure (Inspection) in MySQL

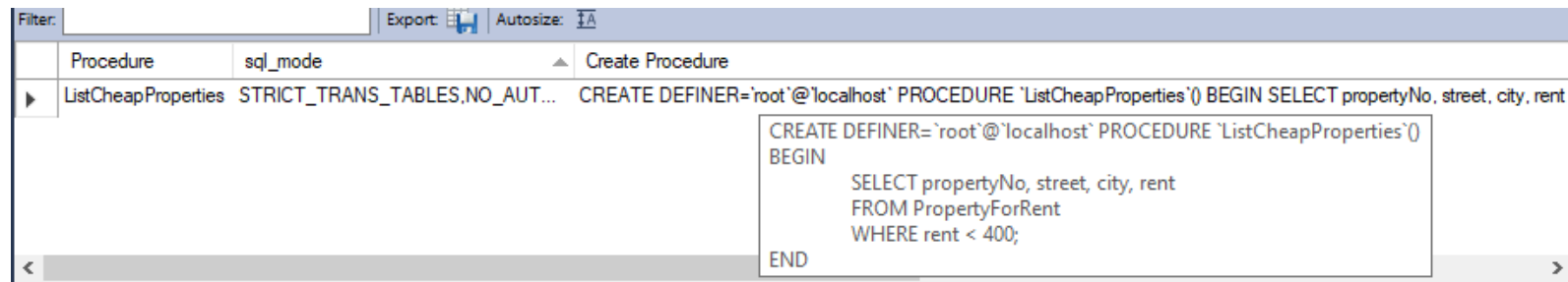
- List all procedures:

SHOW PROCEDURE STATUS WHERE db = 'dreamhome';

- View the Source Code:

SHOW CREATE PROCEDURE <procedure_name>;

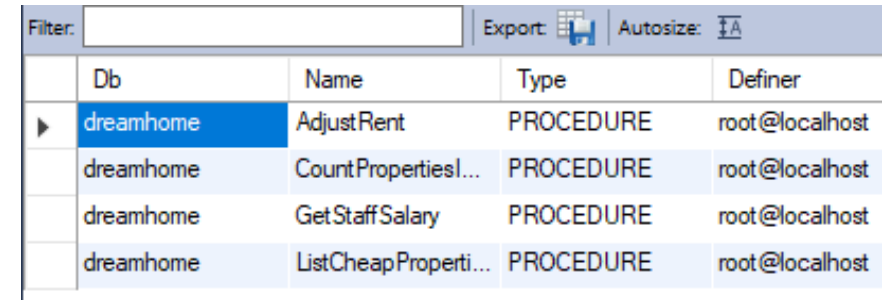
Example:



The screenshot shows the MySQL Workbench interface. The top toolbar includes a 'Filter' input, an 'Export' button, and an 'Autosize' button. Below the toolbar, a table lists procedures. The 'ListCheapProperties' procedure is selected, and its source code is displayed in a text area. The code is as follows:

```
CREATE DEFINER='root'@'localhost' PROCEDURE `ListCheapProperties`()
BEGIN
    SELECT propertyNo, street, city, rent
    FROM PropertyForRent
    WHERE rent < 400;
END
```

Example:



The screenshot shows the results of the 'SHOW PROCEDURE STATUS' query. The table has five columns: Db, Name, Type, and Definer. The results are as follows:

Db	Name	Type	Definer
dreamhome	AdjustRent	PROCEDURE	root@localhost
dreamhome	CountPropertiesI...	PROCEDURE	root@localhost
dreamhome	GetStaffSalary	PROCEDURE	root@localhost
dreamhome	ListCheapProperti...	PROCEDURE	root@localhost

Drop Procedure (removal) in MySQL

DROP PROCEDURE [IF EXISTS] <procedure_name>;

Modes in Procedure Parameters

- In general, each parameter should have a **parameter type** that is **one of the SQL data types**.
- Each parameter should also have a **parameter mode**, which is one of **IN**, **OUT**, or **INOUT**.

IN parameter: It is the default mode. It takes a **parameter as input**, such as an attribute. When we define it, the calling program has to pass an argument to the stored procedure.

OUT parameters: It is used to pass a **parameter as output**. Its value can be changed inside the stored procedure, and the changed (new) value is passed back to the calling program. It is noted that a procedure cannot access the OUT parameter's initial value when it starts.

INOUT parameters: It is a **combination of IN and OUT parameters**. It means the calling program can pass the argument, and the procedure can modify the INOUT parameter, and then pass the new value back to the calling program.

Without parameters

-- 1. Change the delimiter from ; to //

DELIMITER //

CREATE PROCEDURE ListCheapProperties()

BEGIN

SELECT propertyNo, street, city, rent

FROM PropertyForRent

WHERE rent < 400;

END //

-- 2. Change the delimiter back to ;

DELIMITER ;

Scenario: Retrieve "low-performing" properties (those with a rent below 400) and display a simple list of their details.

After running the code above, use this statement to see if it worked:

CALL ListCheapProperties();

Result:

Filter:		Export: 		Autosize: 	
	propertyNo	street	city	rent	
▶	PG36	2 Manor Rd	Glasgow	375.00	
	PG4	6 Lawrence St	Glasgow	350.00	

IN parameter

***Scenario:** Count how many properties are available in a specific city.*

DELIMITER //

CREATE PROCEDURE CountPropertiesInCity (**IN** searchCity VARCHAR(15))

BEGIN

DECLARE propCount **INT**;

SELECT COUNT(*) INTO propCount

FROM PropertyForRent

WHERE city = searchCity;

-- Logic to display or use propCount

SELECT propCount **AS** TotalProperties;

END//

DELIMITER ;

After running the code above, use this statement to see if it worked:

CALL CountPropertiesInCity('Glasgow');

Result:

Filter:	
TotalProperties	
▶	4

OUT parameter

***Scenario:** Retrieve the salary of a specific staff member to use in another calculation.*

DELIMITER //

```
CREATE PROCEDURE GetStaffSalary (IN sNo VARCHAR(5), OUT sSalary DECIMAL(7,2))
```

```
BEGIN
```

```
    SELECT salary INTO sSalary
```

```
    FROM Staff
```

```
    WHERE staffNo = sNo;
```

```
END//
```

DELIMITER ;

When you call the procedure, the OUT parameter tells the database systems that its value goes out of the procedure. Now, we will pass its value to a session variable **@currentSalary** in the **CALL** statement as given.

After running the code above, use this statement to see if it worked:

```
CALL GetStaffSalary('SG37', @currentSalary);  
SELECT @currentSalary;
```

Result:

Filter:	
	@currentSalary
▶	12000.00

INOUT parameter

***Scenario:** Apply a percentage increase to a property's rent. The procedure takes the current rent (via the table), applies a multiplier, and returns the new value.*

DELIMITER //

```
CREATE PROCEDURE AdjustRent (INOUT amount DECIMAL(7,2), IN increaseRatio REAL)
```

```
BEGIN
```

```
-- amount enters as the current rent
```

```
SET amount = amount * (1 + increaseRatio);
```

```
-- amount exits as the new rent
```

```
END//
```

DELIMITER ;

INOUT parameter is the **amount**. The **amount** first acts as the **IN** parameter and then **OUT** parameter. Therefore, we can call it the **INOUT** parameter mode.

After running the code above, use this statement to see if it worked:

```
SET @rentValue = 500.00;
```

```
CALL AdjustRent(@rentValue, 0.10); -- 10% increase
```

```
SELECT @rentValue; -- Result will be 550.00
```

Result:

Filter:
@rentValue
▶ 550.00

Declarations in Procedures

- **Variables** must be declared at the very beginning of the **BEGIN...END** block, before any other SQL statements.

```
DECLARE variable_name datatype(size) [DEFAULT default_value];
```

Examples:

```
DECLARE currency CHAR(3) DEFAULT 'USD';
```

```
DECLARE totalSale DEC(10,2);
```

```
DECLARE stockCount INT DEFAULT 0;
```

- If no **DEFAULT** is provided, the value is automatically initialized to **NULL**.
- Note that it is possible to declare a variable as NOT NULL, although in this case an initial value must be assigned to the variable.

Example: **DECLARE** v_staff_id **VARCHAR**(5) **NOT NULL** **DEFAULT** 'S0001';

Declarations in Procedures

- To assign a variable a value, use the **SELECT INTO** statement to assign the result of a query to a variable.

Example:

```
DELIMITER //
```

```
CREATE PROCEDURE CountPropertiesInCity (IN searchCity VARCHAR(15))
```

```
BEGIN
```

```
    DECLARE propCount INT;
```

```
    SELECT COUNT(*) INTO propCount
```

```
    FROM PropertyForRent
```

```
    WHERE city = searchCity;
```

```
    SELECT propCount AS 'Number of Properties';
```

```
END//
```

```
DELIMITER ;
```

Activity 1

Design a stored procedure named `AnalyzePosition` that accepts a specific job position (e.g., 'Manager', 'Assistant') and returns the total count of staff members in that role.

```
DELIMITER //
CREATE PROCEDURE AnalyzePosition (IN p_jobTitle VARCHAR(15))
BEGIN
    -- 1. Declare a local variable to hold the count
    DECLARE staffCount INT;
    -- 2. Calculate the count based on the IN parameter
    SELECT COUNT(*) INTO staffCount FROM Staff WHERE position = p_jobTitle;
    -- 3. Display the result with a meaningful header
    SELECT CONCAT('There are ', staffCount, ' employees with the title: ', p_jobTitle) AS 'Position Report';
END //
DELIMITER ;
```

-- Test for Managers

```
CALL AnalyzePosition('Manager');
```

Activity 2

Design a stored procedure named `SimulateRentIncrease` that takes a property's current rent, applies a percentage increase, and updates that same variable with the new total.

```
DELIMITER //
CREATE PROCEDURE SimulateRentIncrease (INOUT io_rent DECIMAL(10,2), IN p_percentage DECIMAL(4,2))
BEGIN
    -- 1. Perform the calculation. We take the current value of io_rent and multiply it
    SET io_rent = io_rent * (1 + (p_percentage / 100));
    -- 2. No final SELECT is needed inside the procedure. because the value is stored back in the variable itself.
END //
DELIMITER ;
```

Control Statements

Conditional Statement - IF

- The IF statement allows you to evaluate one or more conditions and execute the corresponding code block if the condition is true.
- The IF statement has three forms:
 - `IF...THEN` statement: Evaluate one condition and execute a code block if the condition is true.
 - `IF...THEN...ELSE` statement: Evaluate one condition and execute a code block if the condition is true; otherwise, execute another code block.
 - `IF...THEN...ELSEIF...ELSE` statement: Evaluate multiple conditions and execute a code block if a condition is true. If all conditions are false, execute the code block in the ELSE branch.

IF...THEN

Scenario: Check if a property's rent is extremely high and, if so, log a warning message.

```
DELIMITER //
CREATE PROCEDURE CheckHighRent(IN pNo VARCHAR(5))
BEGIN
    DECLARE currentRent DECIMAL(7,2);
    SELECT rent INTO currentRent FROM PropertyForRent WHERE propertyNo = pNo;
    IF currentRent > 2000 THEN
        SELECT 'Warning: This property has a premium rent' AS Status;
    END IF;
END //
DELIMITER ;
```

IF...THEN...ELSE (Two-Way Branching)

Scenario: Determine if a staff member is "Senior" based on their salary.

```
DELIMITER //
CREATE PROCEDURE EvaluateStaffLevel(IN sNo VARCHAR(5))
BEGIN
    DECLARE sSalary DECIMAL(7,2);
    SELECT salary INTO sSalary FROM Staff WHERE staffNo = sNo;
    IF sSalary >= 25000 THEN
        SELECT 'Senior Staff' AS Level;
    ELSE
        SELECT 'Junior Staff' AS Level;
    END IF;
END //
DELIMITER ;
```

IF...THEN...ELSEIF...ELSE (Multi-Way Branching)

Scenario: Categorize properties into "Budget," "Standard," or "Luxury" based on their rent.

```
DELIMITER //
CREATE PROCEDURE CategorizeProperty(IN pNo VARCHAR(5))
BEGIN
    DECLARE pRent DECIMAL(7,2);
    SELECT rent INTO pRent FROM PropertyForRent WHERE propertyNo = pNo;
    IF pRent < 400 THEN
        SELECT 'Budget' AS Category;
    ELSEIF pRent >= 400 AND pRent <= 800 THEN
        SELECT 'Standard' AS Category;
    ELSE
        SELECT 'Luxury' AS Category;
    END IF;
END //
DELIMITER ;
```

Activity 3

Design a procedure named *CategorizeRent* that takes a property's monthly rent as an **IN** parameter and returns a descriptive "market status" as an **OUT** parameter based on the following rules:

Budget: Rent is less than 500.

Standard: Rent is between 500 and 1500.

Premium: Rent is between 1501 and 3000.

Luxury: Rent is anything above 3000.

```
DELIMITER //
CREATE PROCEDURE CategorizeRent ( IN p_rent
DECIMAL(10,2), OUT p_status VARCHAR(20))
BEGIN
IF p_rent < 500 THEN
    SET p_status = 'Budget';
ELSEIF p_rent >= 500 AND p_rent <= 1500 THEN
    SET p_status = 'Standard';
ELSEIF p_rent > 1500 AND p_rent <= 3000 THEN
    SET p_status = 'Premium';
ELSE
    SET p_status = 'Luxury';
END IF;
END //
DELIMITER ;
```

Conditional Statement - CASE

- Besides the **IF** statement, MySQL provides an alternative conditional statement called the **CASE** statement, used in stored procedures.
- The CASE statements make the code more readable and efficient.
- The CASE statement has two forms:

Simple CASE statement

Searched CASE statement

CASE

Simple CASE (Value Matching)

Scenario: Convert the *type code* of a property into a full descriptive word for a report.

```
DELIMITER //
CREATE PROCEDURE GetPropertyDescription(IN pType
CHAR(1))
BEGIN
    DECLARE description VARCHAR(20);
    CASE pType
        WHEN 'H' THEN SET description = 'House';
        WHEN 'F' THEN SET description = 'Flat';
        WHEN 'B' THEN SET description = 'Bungalow';
        ELSE SET description = 'Unknown';
    END CASE;
    SELECT description AS 'Property Type Description';
END //
DELIMITER ;
```

Searched CASE (Range/Logic Matching)

Scenario: Assign a "Service Tier" to a property based on its monthly rent.

```
DELIMITER //
CREATE PROCEDURE AssignServiceTier(IN pNo VARCHAR(5))
BEGIN
    DECLARE pRent DECIMAL(7,2);
    DECLARE tier VARCHAR(10);
    SELECT rent INTO pRent FROM PropertyForRent WHERE
propertyNo = pNo;
    CASE
        WHEN pRent >= 1000 THEN SET tier = 'Premium';
        WHEN pRent >= 500 THEN SET tier = 'Standard';
        WHEN pRent > 0 THEN SET tier = 'Economy';
        ELSE SET tier = 'Invalid';
    END CASE;
    SELECT pNo AS Property, tier AS 'Service Tier';
END //
DELIMITER ;
```

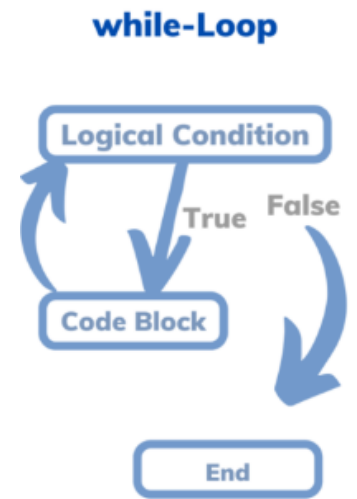
Conditional Statement - LOOP

- The LOOP statement allows you to **execute one or more statements repeatedly**.
- The LOOP can have **optional labels** at the beginning and end of the block.
- You can use,
 - ITERATE** statement to skip the current iteration and start a new one
 - LEAVE** statement to immediately terminate the loop and move to the next statement after the END block.
- Distinct types of loops:
 - WHILE**: A pre-test loop (checks the condition at the start).
 - REPEAT**: A post-test loop (checks the condition at the end).
 - LOOP**: A simple loop with no built-in condition (requires LEAVE to exit).

WHILE Loop

- Checks a condition **at the beginning**. If the condition is true, the code inside runs; if it is false, the loop never starts.

```
DELIMITER //  
CREATE PROCEDURE WeeklyInspectionLog()  
BEGIN  
    DECLARE dayCount INT DEFAULT 1;  
    WHILE dayCount <= 5 DO  
        SELECT CONCAT('Property Inspected on Day ', dayCount) AS 'Log Entry';  
        -- Move to the next day  
        SET dayCount = dayCount + 1;  
    END WHILE;  
    SELECT 'All inspections for the week are complete.' AS Status;  
END //  
DELIMITER ;
```



```
WHILE <condition> DO  
    <statements>;  
END WHILE;
```

CALL WeeklyInspectionLog();

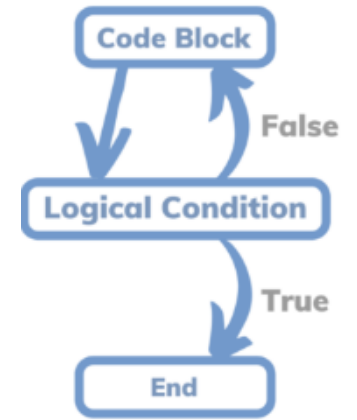
Results:

- Property Inspected on Day 1
- Property Inspected on Day 2
- Property Inspected on Day 3
- Property Inspected on Day 4
- Property Inspected on Day 5

REPEAT Loop

- Checks the condition **at the end**. This means the code inside will always run **at least once**. Note that the loop stops when the condition becomes true (UNTIL logic).

repeat-Loop



```
DELIMITER //  
CREATE PROCEDURE SecurityKnock()  
BEGIN  
    DECLARE knocks INT DEFAULT 0;  
    REPEAT  
        -- Action: Perform the knock  
        SET knocks = knocks + 1;  
        SELECT CONCAT('Knock #', knocks) AS 'Security Action';  
        -- Check: Stop once we have knocked 3 times  
    UNTIL knocks = 3  
    END REPEAT;  
    SELECT 'Entering the property...' AS Status;  
END //  
DELIMITER ;
```

CALL SecurityKnock();

Results:

- Knock #1
- Knock #2
- Knock #3
- Entering the property...

REPEAT

<statements>;

UNTIL <condition>

END REPEAT;

Simple Loop

- The LOOP statement does **not have a built-in condition check**. It will run forever unless you manually stop it using the **LEAVE** statement. To use LEAVE, you must give the loop a **label**.

```
loop_label: LOOP  
    <statements>;  
    IF <condition> THEN  
        LEAVE loop_label;  
    END IF;  
END LOOP;
```

```
DELIMITER //  
CREATE PROCEDURE ArchiveOldViewings()  
BEGIN  
    DECLARE recordsProcessed INT DEFAULT 0;  
    archiving_process: LOOP // Labeling the loop 'archiving_process'  
        -- 1. Manual Exit Check: Stop once we reach 5 records  
        IF recordsProcessed = 5 THEN  
            LEAVE archiving_process;  
        END IF;  
        -- INSERT and DELETE statements here.  
        SET recordsProcessed = recordsProcessed + 1;  
        SELECT CONCAT('Record #', recordsProcessed, ' successfully archived.') AS Progress;  
    END LOOP archiving_process;  
    SELECT 'Archiving complete. 5 records moved.' AS FinalStatus;  
END //  
DELIMITER ;
```

Scenario: The manager wants to move the 5 oldest viewing records into an Archive_Viewing table to keep the main table fast.

Error Handling

Introduction

- **Condition Handler:** A specific block of code designed to catch [errors](#) or [warnings](#) that occur during the execution of a stored procedure.
 - In MySQL, **conditions** refer to [errors, warnings, or exceptional cases that require proper handling](#).
- Without handlers, a SQL error causes the procedure to crash immediately.
- When a **condition** arises during the execution of a stored procedure, you should handle it properly, such as [exiting](#) or [recovering and continuing](#) the current code block with a meaningful message.
- To handle a **condition**, you declare a handler using the **DECLARE ... HANDLER** statement.

```
DECLARE [Action] HANDLER FOR [Condition]
BEGIN
    -- Code to execute when the error happens
END;
```

Introduction

```
DECLARE [Action] HANDLER FOR [Condition]
BEGIN
    -- Code to execute when the error happens
END;
```

[Action]: What happens after the **handler** runs?

- This declares a handler, instructing whether it should exit or continue the enclosing stored procedure when a specified condition occurs.
 - **EXIT:** The **handler** executes its code, and the **procedure terminates** immediately. It does not finish the remaining lines.
 - **CONTINUE:** The **handler** executes its code, and the procedure resumes at **the very next statement** after the one that caused the error.

[Condition]: What specific error are we watching for? (e.g., SQLEXCEPTION, NOT FOUND).

- This specifies the conditions that activate the handler, and you can specify multiple conditions by separating them with commas.

[Body]: The logic to fix the problem or log the error.

- This statement or block of statements executes when the stored procedure encounters one of the specified conditions.

Handlers - Condition Types (Generic)

Condition	Description	SQLSTATE / Identifier
MySQL Error Code	Specific numerical codes used by MySQL internally to identify exact errors.	Example: 1062 (Duplicate entry) or 1048 (Column cannot be null).
SQLSTATE	Standard 5-character alphanumeric string defined by the SQL standard for portability across different DBs.	Example: '23000' (Integrity constraint violation).
NOT FOUND	Used specifically for "empty" results. Triggered when a cursor reaches the end or a SELECT finds nothing.	Corresponds to all codes starting with 02 .
SQLException	A "catch-all" for any general error.	All codes except those starting with 00 (success), 01 (warning), or 02 (not found).
SQLWARNING	Handles minor issues that don't necessarily stop the code from running but might lead to unexpected data.	Corresponds to all codes starting with 01.

Example

Scenario: Adding a new property. If the *propertyNo* already exists, we want to inform the user instead of crashing.

```
CREATE PROCEDURE AddPropertyHandled(IN pNo VARCHAR(5), IN pStreet VARCHAR(25))
```

```
BEGIN
```

```
-- 1. Setup the safety net
```

```
DECLARE EXIT HANDLER FOR 1062
```

Use specific error codes (like 1062) for known issues and SQLEXCEPTION as a final backup.

```
BEGIN
```

```
    SELECT 'USER ERROR: That Property Number is already in use.' AS Message;
```

```
END;
```

```
-- 2. Attempt the action
```

```
INSERT INTO PropertyForRent (propertyNo, street)
```

```
VALUES (pNo, pStreet);
```

```
SELECT 'Property added successfully!' AS Status;
```

```
END;
```

Handlers must be declared **after** variables but **before** the main logic of the procedure.

Example (Cont...)

```
CREATE PROCEDURE AddPropertyHandled(IN pNo VARCHAR(5), IN pStreet VARCHAR(25))
BEGIN
    -- 1. Setup the safety net
    DECLARE EXIT HANDLER FOR 1062
    BEGIN
        SELECT 'USER ERROR: That Property Number is already in use.' AS Message;
    END;
    -- 2. Attempt the action
    INSERT INTO PropertyForRent (propertyNo, street)
    VALUES (pNo, pStreet);
    SELECT 'Property added successfully!' AS Status;
END;
```

Scenario 1: Successful Insertion: Provide a **new** property number that does not exist in the PropertyForRent table.

-- Executing the procedure with a new ID
CALL AddPropertyHandled('PG99', '15 High St');

Result: Property added successfully!

Scenario 2: Triggering the EXIT Handler (Duplicate Key)

Insert a property number that is **already assigned** to another house.

-- Executing the procedure with an existing ID (e.g., PG4)
CALL AddPropertyHandled('PG4', '6 Lawrence St');

Result: USER ERROR: That Property Number is already in use.

Error Handling - Named Condition (Specific)

- Use MySQL **DECLARE ... CONDITION** to associate a [name](#) with a condition specified by a [MySQL error code](#) or [SQLSTATE](#) value before you actually use it in a handler.
 - **MySQL Error Code:** A number specific to MySQL (e.g., 1062 for a duplicate entry).
 - **SQLSTATE:** A 5-character alphanumeric string that is standardized across many database systems (e.g., '23000' for a constraint violation).
- Named conditions make your store procedures code more readable and expressive.

```
DECLARE condition_name CONDITION FOR condition_value;
```

- First, specify the condition's name
- Second, provide the condition value. The condition value can be a [MySQL error code](#) or a [SQLSTATE](#) value.

Example

Instead of writing a handler for the number 1062, you can name it `Duplicate_Property_ID` to make the code clearer.

```
DELIMITER //
CREATE PROCEDURE AddPropertyNamed(IN pNo VARCHAR(5), IN pStreet VARCHAR(25))
BEGIN
    -- 1. DECLARE the named condition
    DECLARE Duplicate_Property_ID CONDITION FOR 1062;
    -- 2. Use that name in the HANDLER (Clearer logic)
    DECLARE EXIT HANDLER FOR Duplicate_Property_ID
    BEGIN
        SELECT 'ERROR: This Property ID is already taken.' AS Message;
    END;
    -- 3. The actual task
    INSERT INTO PropertyForRent (propertyNo, street) VALUES (pNo, pStreet);
    SELECT 'Property added successfully!' AS Status;
END //
DELIMITER ;
```

SIGNAL Statement

- The MySQL **SIGNAL** statement is used to convey error information in stored procedures.
- It ensures that exceptions are properly handled, preventing sudden procedure termination.
- You can use the **DECLARE ... HANDLER** statement to effectively manage exceptions in MySQL. It allows you to specify how different types of exceptions should be handled within a stored procedure. By using this statement in conjunction with SIGNAL, you can enable precise control over error handling.
- The **SIGNAL** statement allows for the customization of error messages using the **SET MESSAGE_TEXT** command.
- This is helpful for modifying error messages to provide more meaningful feedback to handlers, applications, or clients.

SIGNAL *condition_value* [**SET** *signal_information_item*]

Example

DELIMITER //

CREATE PROCEDURE `update_salary`(IN p_employee_id INT, IN p_salary DECIMAL(10,2))
BEGIN

DECLARE employee_count INT;

-- Named Conditions for better readability

DECLARE NOT_FOUND_CONDITION CONDITION FOR SQLSTATE '45000';

DECLARE INVALID_DATA_CONDITION CONDITION FOR SQLSTATE '45000';

-- Check if employee exists

SELECT COUNT(*) INTO employee_count **FROM** employees **WHERE** id = p_employee_id;

IF employee_count = 0 **THEN**

-- Using the named condition to make the error clear

SIGNAL NOT_FOUND_CONDITION SET MESSAGE_TEXT = 'Employee not found';

END IF;

-- Validate salary

IF p_salary < 0 **THEN**

SIGNAL INVALID_DATA_CONDITION SET MESSAGE_TEXT = 'Salary cannot be negative';

END IF;

-- Final update if everything is fine

UPDATE employees **SET** salary = p_salary **WHERE** id = p_employee_id;

SELECT 'Salary updated successfully' **AS** Status;

END //

DELIMITER ;

CALL `update_salary`(1, 7000);

Result:

ERROR 1644 (45000):

Employee not found

CALL `update_salary`(1,-7000);

Result:

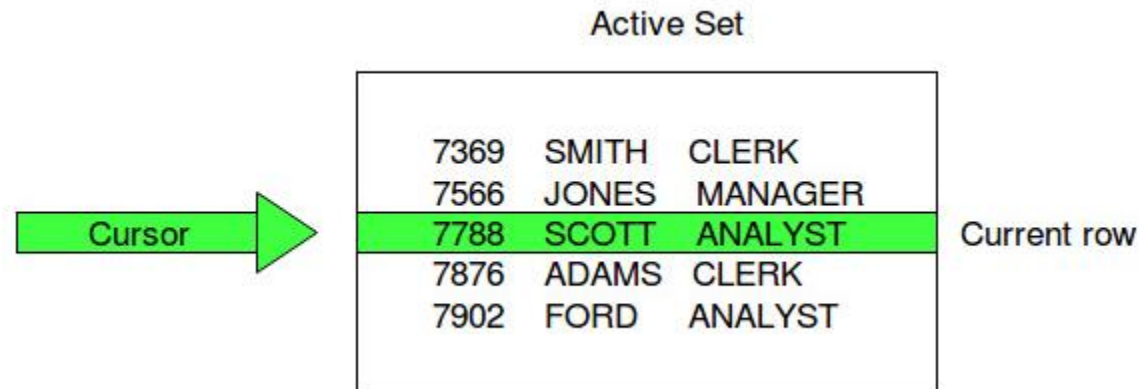
ERROR 1644 (45000): Salary

cannot be negative

CURSORS

Introduction

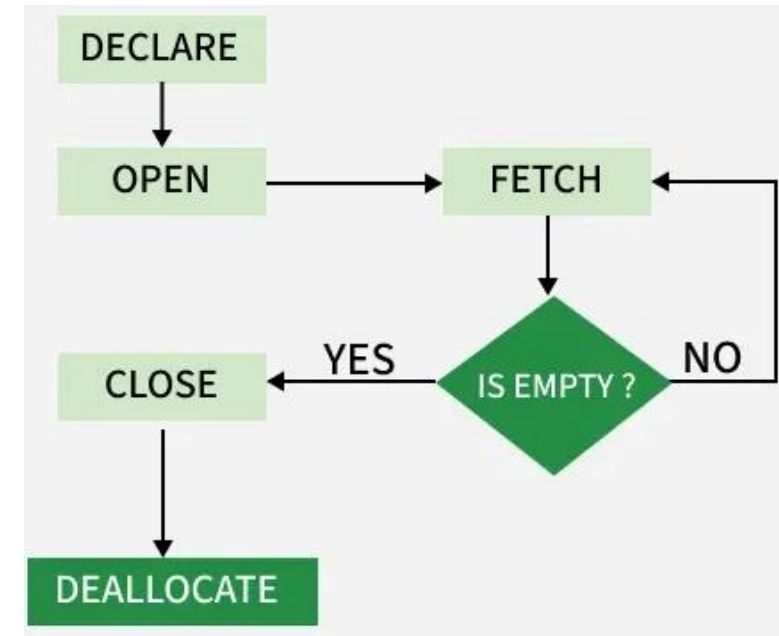
- A SELECT statement can return an arbitrary number of rows (that is, zero, one, or more rows).
- A **cursor** is a database object that iterates the result of a SELECT statement and allows the rows of a query result to be accessed one at a time.
- A **cursor** acts as a pointer to a particular row of the query result.



- Use cursors within stored procedures, triggers, and functions where you need to process individual rows returned by a query one at a time.

Introduction

- A **cursor** must be *declared* and *opened* before it can be used.
- Once the cursor has been opened, the rows of the query result can be retrieved one at a time using a **FETCH** statement.
- Each time you call the **FETCH** statement, the cursor attempts to read the next row in the result set.
- When the cursor reaches the end of the result set, it will not be able to retrieve the data, and a condition is raised. The handler is used to handle this condition.
- When working with a MySQL cursor, you must also declare a **NOT FOUND** handler to manage the situation when the cursor cannot find any row.
- A cursor must be *closed* to deactivate it after it is no longer required.



The Standard Cursor Lifecycle

Syntax

Define the cursor and the SELECT statement it will use (Identify *active set*). This must be done after variable declarations but before any other logic.

-- declare a cursor

```
DECLARE cursor_name CURSOR FOR  
    SELECT column1, column2  
    FROM your_table  
    WHERE your_condition;
```

-- open the cursor

```
OPEN cursor_name;  
FETCH cursor_name INTO variable1, variable2;
```

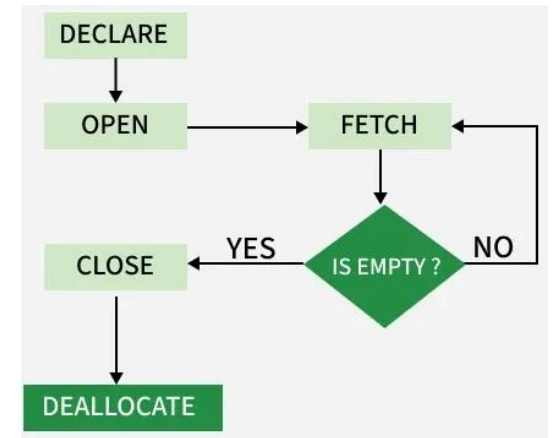
-- process the data

-- close the cursor

```
CLOSE cursor_name;
```

Initialize the cursor and execute the query to populate the result set.

Deallocate the cursor and release the memory it was using.



Retrieves the current row values into output variables using the **FETCH INTO** statement.

Each **FETCH** statement advances the pointer to the next row of the **active set**.

Example

```
DELIMITER //
```

```
CREATE PROCEDURE CalculateStaffBonus()
```

```
BEGIN
```

```
    DECLARE s_name VARCHAR(50);
```

```
    DECLARE s_salary DECIMAL(10,2);
```

```
    DECLARE done INT DEFAULT 0;
```

```
    -- Declare the CURSOR
```

```
    DECLARE staff_cursor CURSOR FOR
```

```
        SELECT fName, salary FROM Staff;
```

```
    -- Declare a NOT FOUND handler to stop the loop
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
    OPEN staff_cursor; // Open the cursor
```

```
    -- Create a loop to FETCH data
```

```
    read_loop: LOOP
```

```
        FETCH staff_cursor INTO s_name, s_salary;
```

```
        IF done THEN
```

```
            LEAVE read_loop;
```

```
        END IF;
```

```
        -- Process the data (Example: 10% bonus)
```

```
        SELECT CONCAT(s_name, ' receives a bonus of  
£', s_salary * 0.10) AS BonusStatus;
```

```
    END LOOP;
```

```
    -- CLOSE the cursor
```

```
    CLOSE staff_cursor;
```

```
END //
```

```
DELIMITER ;
```

Important Rules for Cursors

- **Order of Declaration:** Variables must be declared first, then Conditions, then Cursors, and finally Handlers.
- **Read-Only:** In MySQL, cursors are "read-only," meaning you cannot update the table directly through the cursor pointer.
- **Non-scrollable:** You can only move forward through the rows; you cannot skip rows or go back to a previous one.
- **FETCH** variables (`current_staff_id`, `current_salary`) must match the **data types** and **order** of the columns in your **SELECT** statement perfectly.

Stored Functions

Creating a Function

- A **Stored Function** is a database routine designed to perform a calculation or transformation and return a **single value**.
- While Procedures are used for **doing** (actions), Functions are used for **asking** (calculations).
- The general form of declaring a function is as follows:

```
CREATE FUNCTION function_name (parameter_name data_type)  
    RETURNS return_data_type  
    [DETERMINISTIC | NOT DETERMINISTIC]  
    BEGIN  
        <local declarations>  
        <function body> ;  
        RETURN results;  
    END;
```

* A **return type** is necessary.

Creating a Function

```
CREATE FUNCTION function_name (parameter_name data_type)  
    RETURNS return_data_type  
    [DETERMINISTIC | NOT DETERMINISTIC]  
    BEGIN  
        <local declarations>  
        <function body> ;  
        RETURN results;  
    END;
```

RETURNS clause: Declares the data type of the value to be returned.

RETURN statement: The specific command that sends the value back and ends the function.

DETERMINISTIC: A required characteristic in many configurations; it tells MySQL that the function will always return the same result for the same input.

Creating a Function

MySQL uses binary logging to keep replicas (slave servers) in sync with the master.

DETERMINISTIC

- Tells MySQL that if you give the function the same input (e.g., $10 + 10$), it will always produce the same output (20).
- This is safe to repeat on a replica server.
- Always include DETERMINISTIC for simple math/logic functions.

NOT DETERMINISTIC

- Means the function might return different results even with the same input (e.g., a function that uses NOW() or RAND()).
- This could cause the data on a replica server to become different from the master.

Example

Scenario: A manager wants a score (1-10) for staff members based on how many properties they manage.

```
DELIMITER //
CREATE FUNCTION get_efficiency_score(p_count INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    IF p_count > 10 THEN
        RETURN 'High Efficiency';
    ELSEIF p_count > 5 THEN
        RETURN 'Standard';
    ELSE
        RETURN 'Low Volume';
    END IF;
END //
DELIMITER ;
```

How to use it in a query:

```
SELECT fName, lName,
get_efficiency_score(managed_properties_count) AS Rating
FROM Staff;
```

Any Questions?